

main two sections:

- head
- body

in body you can have:

- header
- main
- footer

section

lang, dir(rtl, ltr)

p -> for paragraph

h# -> # is a number (smaller number -> bigger header)

*table

- tr -> table row
- th -> table header
- td -> data cell

*list

- ul -> bullet list
- ol -> numbered list
- li -> list elements

a -> link (href="url" target= where this opens)

figure

- img
src="path" alt="a description"
- figcaption

article

form

- label for="that element"
- input id="that element" type

button

type

nav -> navigation links (can be to other html files)

define a class with :

```
<div class="your-class">
```

and style it with:

```
.your-class {  
}
```

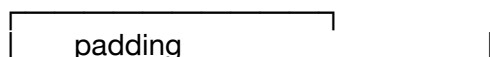
##CSS

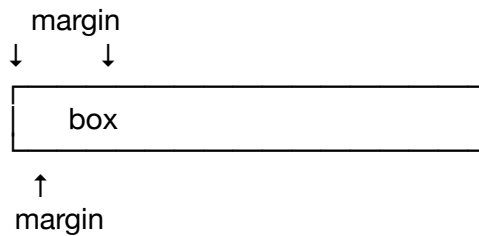
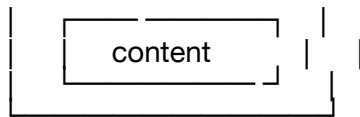
access an element within a class

```
.your-class e {  
}
```

padding vs margin

padding : space between the content and the border in an element
margin: space outside the element





--shorthand : margin : top right bottom left

--border: width style color
 styles: solid, dashes, dotted, double
 radius -> curve

colors:

background-color
 color -> text

font : font-size, font-family, font-weight (: bold, or :700)

alignment:

text-align: left, center, right, justify
 text-decoration (underline, ...)

***display

none -> remove element

flex -> flexible in arrangement

grid ->

grid-template-column: repeat(4, 1fr) -> 4 columns each taking 1 fraction of available space

or you can give custom numbers

grid-template-rows : repeat(auto-fit, minmax(200px, 1fr) -> automatically adjusts

position

static

relative

when you have relative with other attributes like top, right in a class ->

it's relative to itself

when you have just relative in a class -> becomes the reference point for

children -> the child then uses absolute position

absolute -> relative to a positioned ancestor

fixed

sticky

overflow -> what happens when a content doesn't fit

hidden

scroll
auto

img
object-fit : cover, contain,

opacity -> between 0-1

cursor : pointer, default, not-allowed

hover, active, and focus properties(pseudoclasses) for elements

*transitions

can be used on hover effects (in the original class : transition: 0.3s;)

ex:

```
button {  
  background-color: white;  
  transition: background-color 0.3s, transform 0.3s;  
}
```

```
button:hover {  
  background-color: blue;  
  transform: scale(1.1);  
}
```

*transform

translate
rotate
scale
skew
matrix
perspective

***transition

use transition in a class , the animation will occur for the psuedoclass (like :hover, :active)

```
/* apply to one property */  
/* property name | duration | easing function | delay */  
transition: margin-right 4s ease-in-out 1s;
```

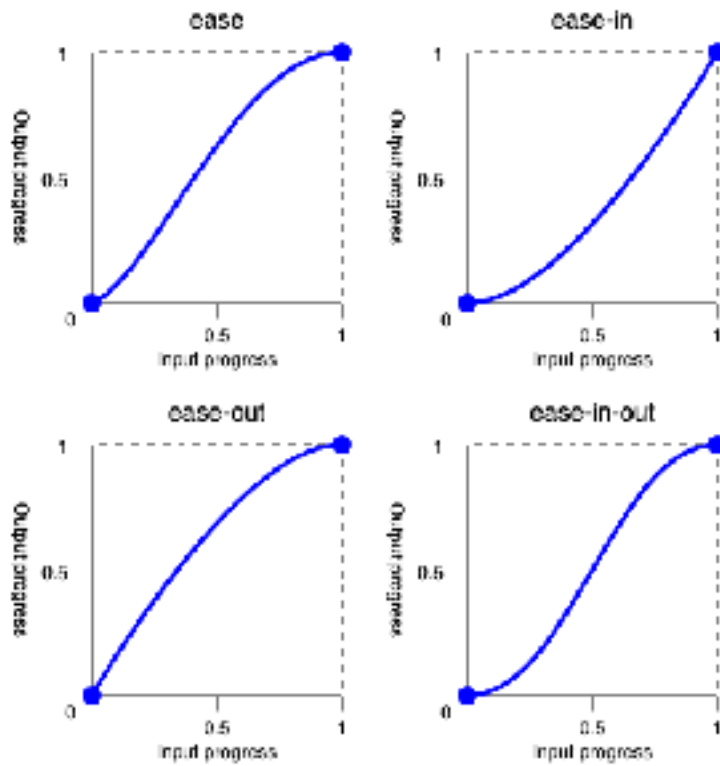
```
/* property name | duration | behavior */  
transition: display 4s allow-discrete;
```

```
/* Apply to 2 properties */  
transition:  
  margin-right 4s,  
  color 1s;
```

```
/* Apply to all changed properties */  
transition: all 0.5s ease-out allow-discrete;  
transition: 200ms linear 50ms;
```

—different ease-functions:

Linear



box-shadow :

/* Four length values and a color */

box-shadow: 2px 2px 2px 1px rgb(0 0 0 / 20%);

offset-x, offset-y, blur-radius, spread-radius — the alpha-value of a color represents the transparency percentage(the higher -> more opaque)

/* inset, length values, and a color */

box-shadow: inset 5em 1em gold;

/* Multiple shadows, separated by commas */

box-shadow:

3px 3px red inset,
-1em 0 0.4em olive;

flexbox:

display : flex

flex : auto -> shares available spaces equally

flex:1 -> this number determines the ratio of children

shorthand => flex: grow shrink basis;

flex-direction: row, column

gap

justify-content -> use to "justify" and align dev boxes

start : left side

end : right side

--depends on the page or DOM direction (RTL, LTR), if the page is ltr then start refers to left and end refers to right.

left : (In flex-direction: column this behaves like start)

right

normal -> is like stretch but it doesn't stretch when column width is set

stretch

space-around , space-evenly, space between

safe , unsafe

justify-content: horizontal alignment

align-items: vertical alignment

the subclass or child of a flex class can have a separate alignment with the rest of the elements in that flex class

--can be done with ---margin-left : auto--- for the element you want to push right

to center a non-flex element :

use box alignment properties:

align-content: center

margin: auto

vw = viewport width -> the % of the width of the viewport

rem = root element's font size

##Responsive design tips:

@media => CSS rule that applies styles only when certain conditions are true (usually based on screen width)

At-rules in CSS -> gives the browser instructions

statement :

@identifier (RULE);

charset, import, namespace, layer

block:

@identifier (RULE) {
}

##pseudo elements

selector::pseudo-element-name {
CSS properties
}

Content pseudo-elements insert or style generated content:

::before - Insert content before an element

::after - Insert content after an element

::marker - Style list item markers

::selection - Style user-selected text

::backdrop - Style dialog backdrop

::selection -> Styles the user-selected text

::spelling-error-> Styles a text that the browser has flagged as incorrectly spelled

::view-transition -> Represents the root of the view transitions overlay, which contains all view transition on the page

::view-transition-group -> Represents a single view transition snapshot group

::view-transition-image-pair -> Represents a container for a view transition's "old" and "new" view states - before and after the transition

::view-transition-new -> Represents the "new" view state of a view transition

::view-transition-old -> Represents the "old" view state of a view transition